**Random Forest** :- when our problem start → Decision tree has **overfitting problem**

Bagging Approach : Bootstrap Aggregation minimize overfitting

$$X_1 = \begin{bmatrix} \cdots x^{(1)} \cdots \\ \cdots x^{(2)} \cdots \\ \cdots x^{(3)} \cdots \\ \cdots x^{(n)} \cdots \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} = f(x_i; \theta) \rightarrow X_2 = \begin{bmatrix} \cdots x^{(1)} \cdots \\ \cdots x^{(4)} \cdots \\ \cdots x^{(3)} \cdots \\ \cdots x^{(n)} \cdots \end{bmatrix}$$

not

B Dataset → Bags $b = \{1, 2, 3 \ldots\ldots B\}$

$f(X_1; \theta_1), f_2(X_2; \theta_2), f_3(X_3; \theta_3) \ldots\ldots f_b(X_b\ \theta_b)$

* Take average of all output

$$f(X; \theta) = \frac{1}{B} \sum_{i=1}^{b} f_i(x_i; \theta_i) \rightarrow \text{Case of Regression}$$

* In Case of Classification Highest Majority Voting

$f(X; \theta) =$ Majority voting $(f_i(x_i; \theta_i), i \in B\{1 \ldots B\}$

$\boxed{b = \text{hyperparameter}}$

1) Getting So many data set → Unrealistic
2) If we have only 1 Dataset → Approximation

* **Problem** $X = \begin{bmatrix} \cdots x^{(1)} \cdots \\ \cdots x^{(2)} \cdots \\ \cdots x^{(3)} \cdots \\ \cdots x^{(n)} \cdots \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(n)} \end{bmatrix}$

*

Bootstraping :- Create B data set from a data set
Some Sample Cows occur 2 or more time.

**Random Forest** :- Can model Non linear Relationship → Collection trees

$D_X = N$ Samples $\quad x^{(i)} \in \mathbb{R}^d$

Step 1 Generate multiple Data set from X (Bootstrapping) B = How many trees wanted to build (hyp)

Step 2 → Build Decision Tree for each Every data set

Step 3 → Instead of evaluating each of every feature Choose a set of feature randomly Use them to build

No of feature for a tree will be Constant → $\sqrt{d}$ → Dry experiment

Agregression
→ $f(x; \theta) = \frac{1}{B} \sum_{i=1}^{B} f_b(x_b; \theta_b)$ ← Majority voting

```
def Randomized-Learn(x, d)
{
  for each node
  {
    features = RandomizeSubset of taken from
    feature split best from feature
  }
  return tree
}
def Rand Forest (X, d)
{ F = ∅
  for b = 1 to B
  {
    Xb = A Bootstrapped dataset from X
    fb = Randomized Learn(Xb, d)
    F = F ∪ {fb}
  }
  return F
```

Teacher's Signature: . . . . . .

The Bootrap agreggation

The 4 Essential Step :- your code need to handle these logic blocks:

1) The Base Learner :- You need a simple model to replicate Usually a simple Decision Trees. → Simple Linear Regression even even a stub or your base

2) Bootstrapping (Sampling with Replacement for each "members" of the ensemble, you must create new training set by randomly picking rows will be picked multiple times, & some won't be picked at all.

3) Parallel Training You train one base learn on each of those bootstrap samples.

4) Aggregation : When it's time to predict, you ask every individual model for its opinion & calculate mean of all their predictions

data, random state, n_estimator